Continuous Integration (CI) Pipeline with Jenkins & Maven/Gradle

Part 1: Jenkins Setup and Configuration

Step 1: Access Jenkins Dashboard

Open a browser and navigate to **localhost:8080**. The Jenkins Welcome screen is displayed with options to create a job or configure distributed builds.

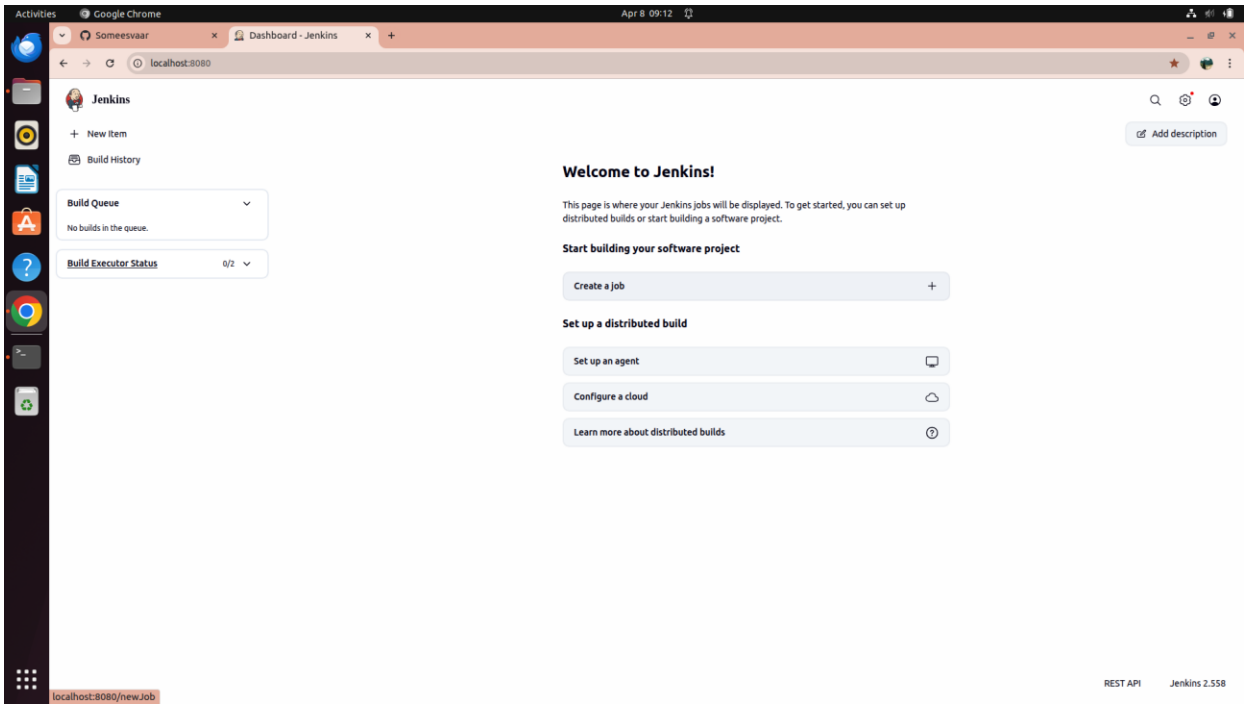

Fig 1: Jenkins Dashboard — Welcome screen at localhost:8080

Step 2: Open Manage Jenkins

Click on the settings gear icon or navigate to **Manage Jenkins** from the Jenkins homepage. This page shows all System Configuration options including Tools, Plugins, Nodes, Security, and Credentials.

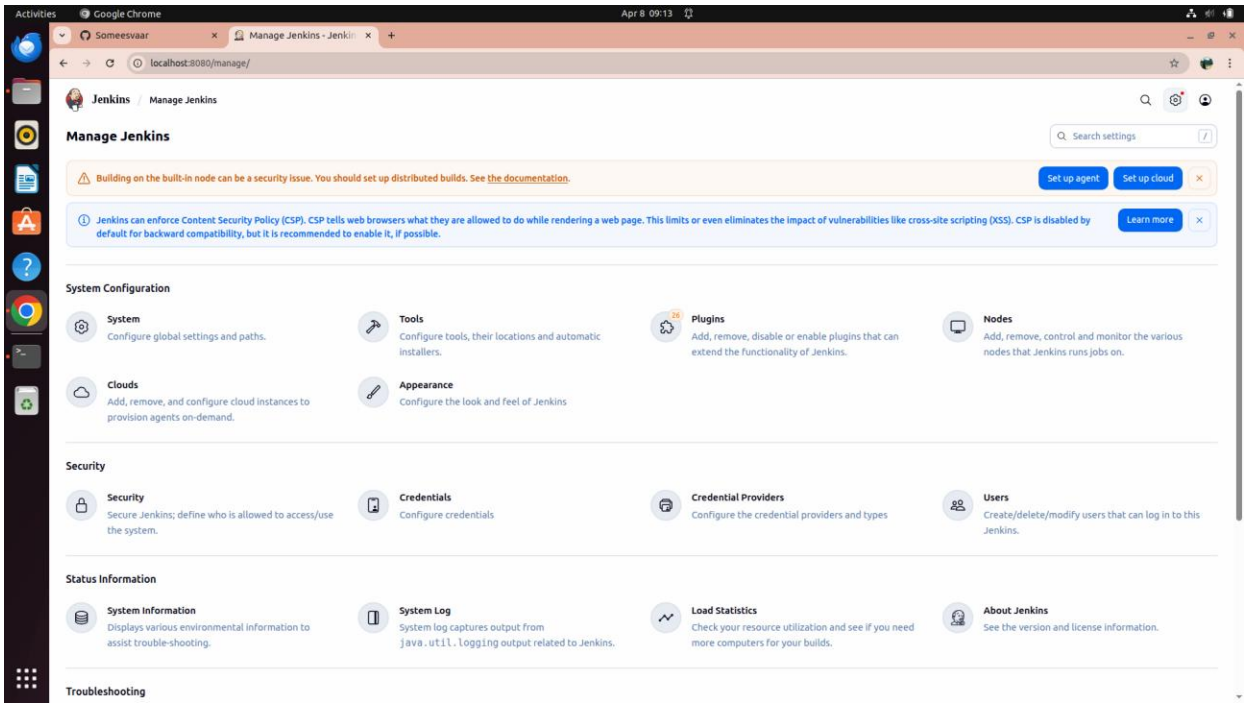

Fig 2: Manage Jenkins page showing System Configuration and Security sections

Step 3: Navigate to Plugins — Check Updates

Go to **Manage Jenkins** → **Plugins**. The Updates tab shows available plugin updates. This is where you install required plugins for the CI pipeline.

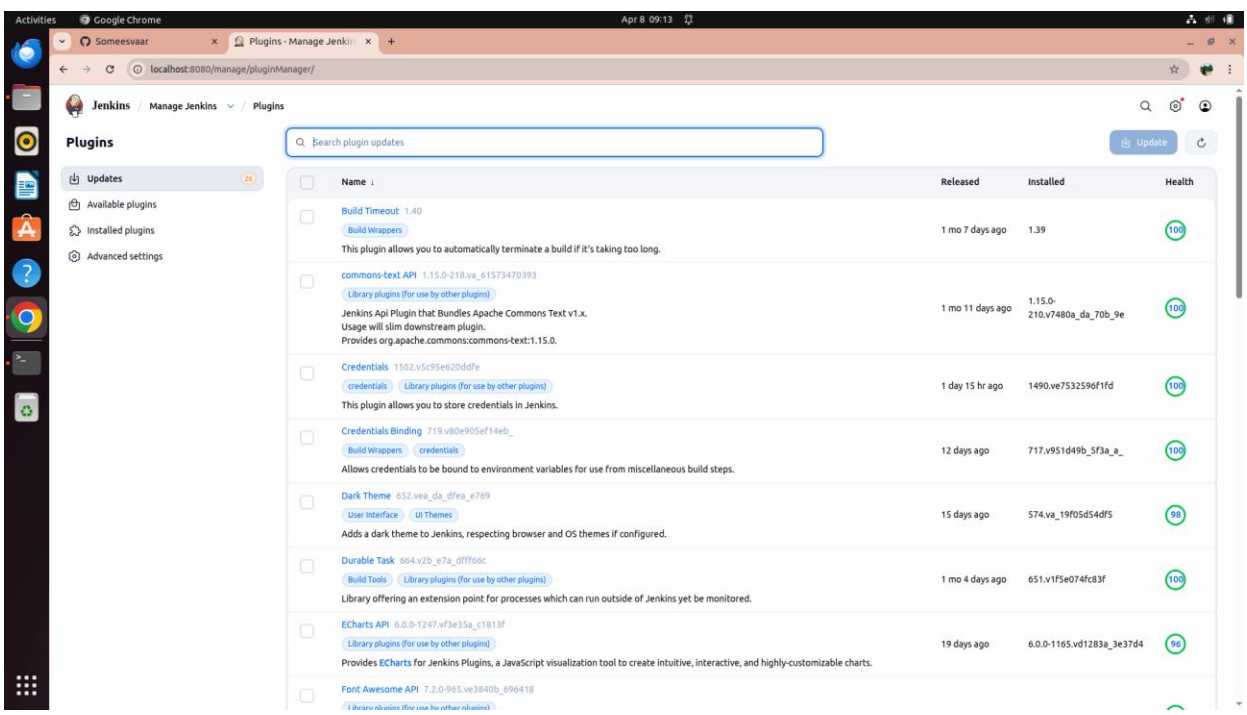

Fig 3: Jenkins Plugins — Updates tab listing available plugin updates

Step 4: Install Pipeline Stage View Plugin

Click on the **Available plugins** tab and search for "pipeline stage". Select **Pipeline: Stage View** and click **Install**. This plugin enables the graphical Stage View on the pipeline job page.

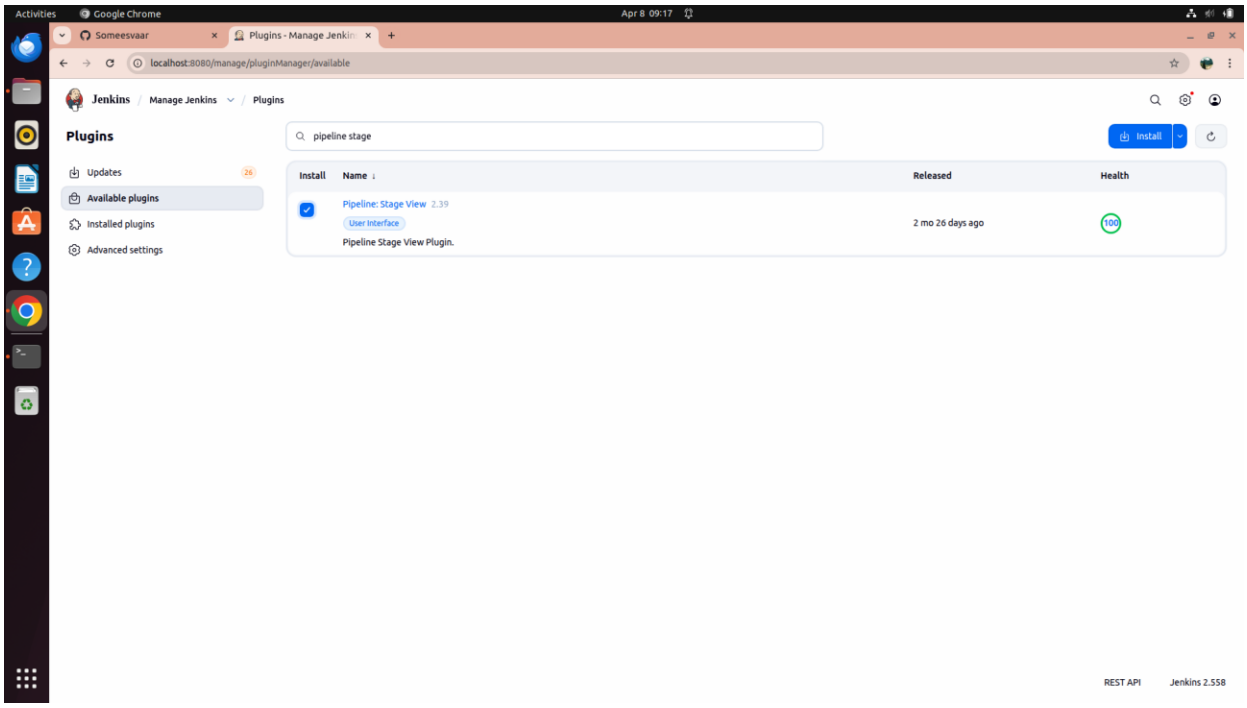

Fig 4: Searching for Pipeline Stage View plugin in Available plugins

Step 5: Pipeline Plugins Install — Download Progress

After clicking Install, the Download Progress page appears. Confirm that **Pipeline Graph Analysis**, **Pipeline: REST API**, **Pipeline: Stage View**, and **Loading plugin extensions** all show **Success**.

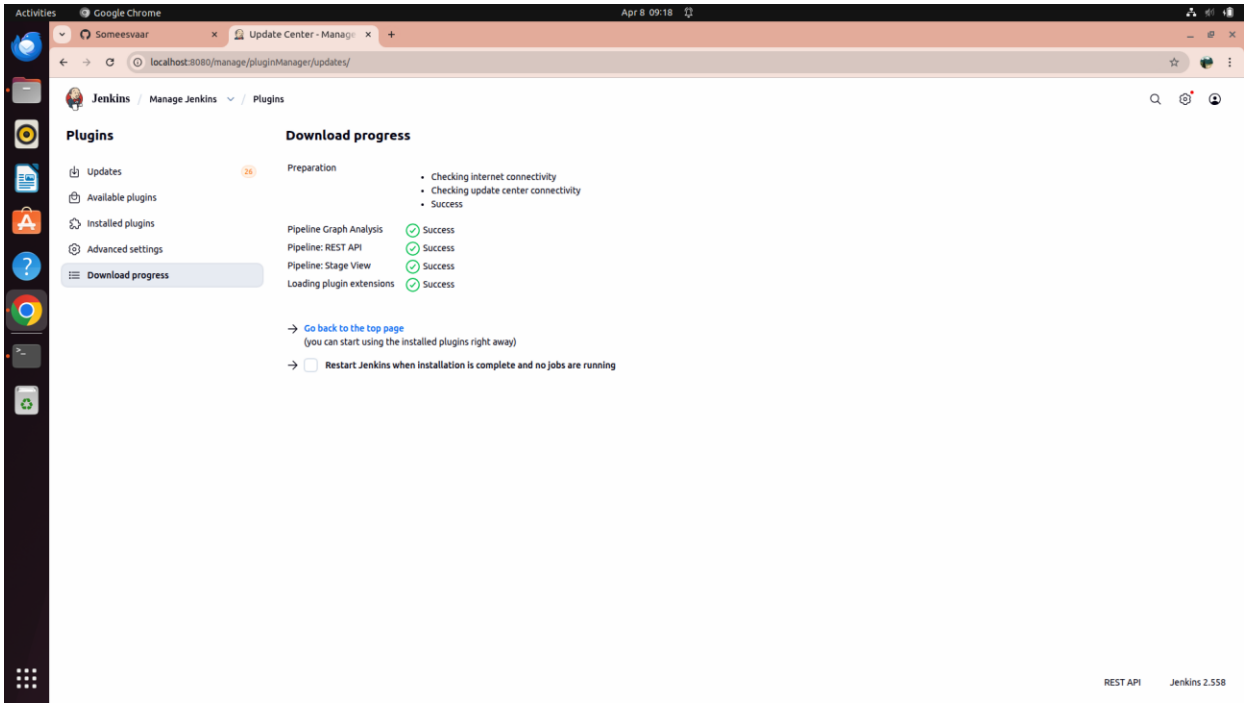

Fig 5: Plugin download progress — all pipeline plugins installed successfully

Step 6: Install GitHub Integration Plugin

Similarly, install the **GitHub Integration** plugin. The Download progress confirms that **GitHub Integration** and **Loading plugin extensions** complete with Success.

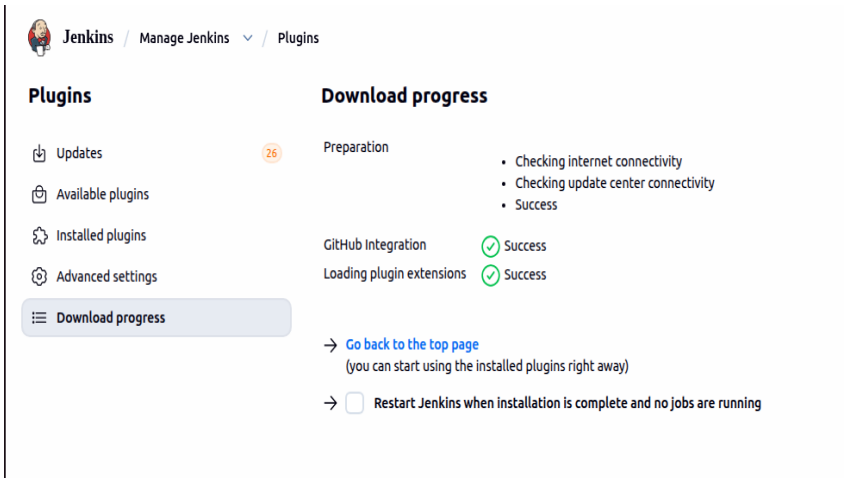

Fig 6: GitHub Integration plugin installed successfully

Step 7: Install Maven Integration Plugin

In the Available plugins tab, search for "maven int" and select **Maven Integration 3.27**. This plugin provides deep integration between Jenkins and Maven, including automatic triggers and automated publisher configuration for JUnit.

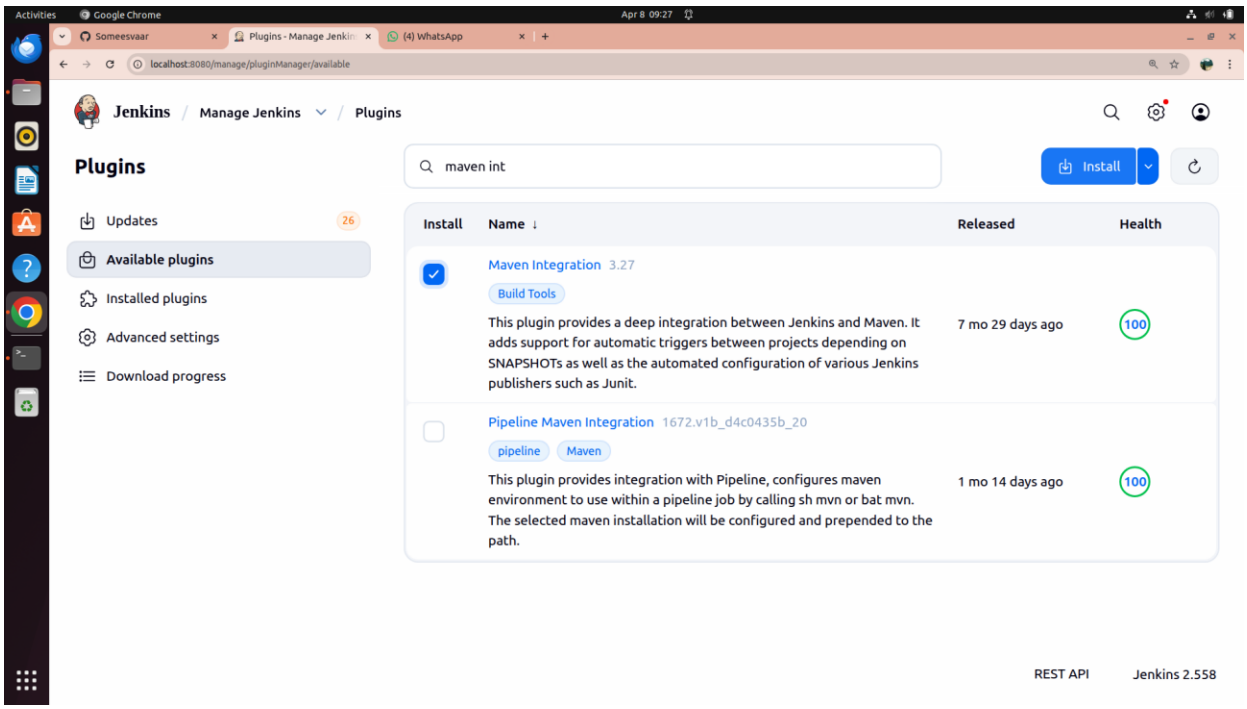

Fig 7: Selecting Maven Integration plugin for installation

Step 8: Maven Integration Plugin — Download Progress

The Download Progress page confirms successful installation of **GitHub Integration**, **Dev Tools Symbols API**, **Javadoc**, **JSch dependency**, and **Maven Integration**. All items show Success.

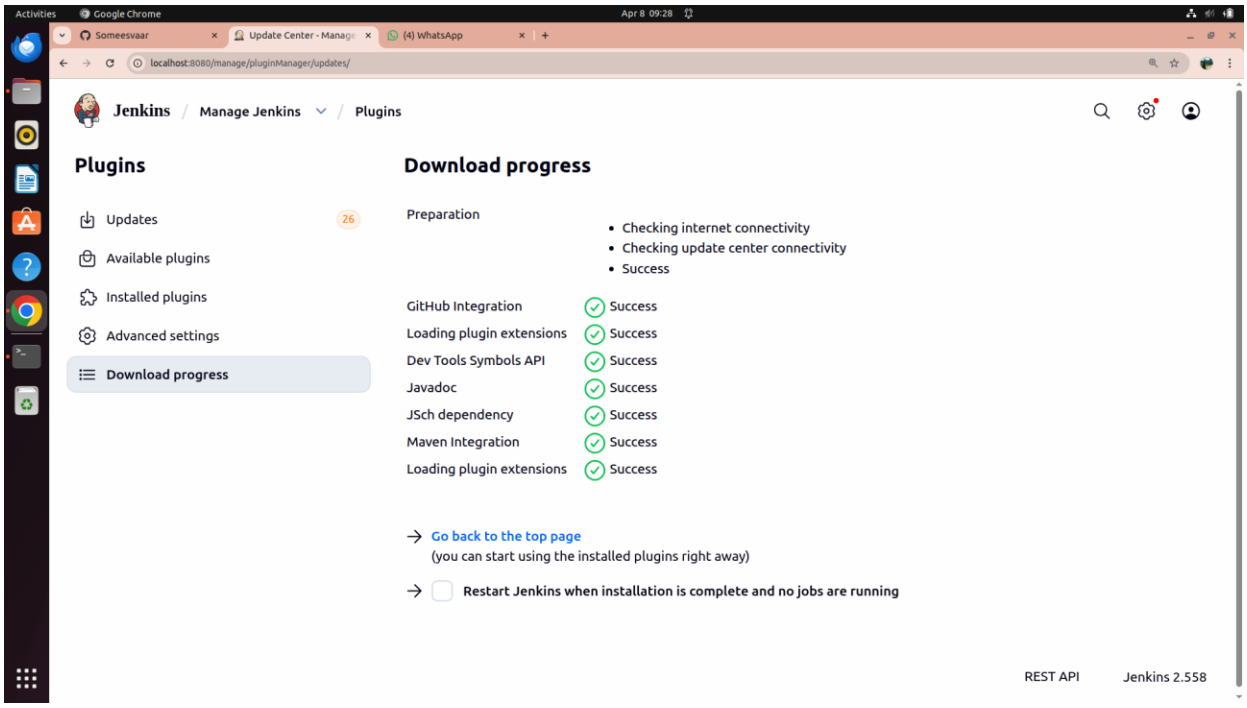

Fig 8: Maven Integration and all dependencies installed successfully

Step 9: Verify Gradle Plugin is Installed

Go to **Installed plugins** and search for "grad". Confirm **Gradle 2.18** is listed with Health 96 and its Enabled toggle is on (blue). This plugin allows Jenkins to invoke Gradle build scripts directly.

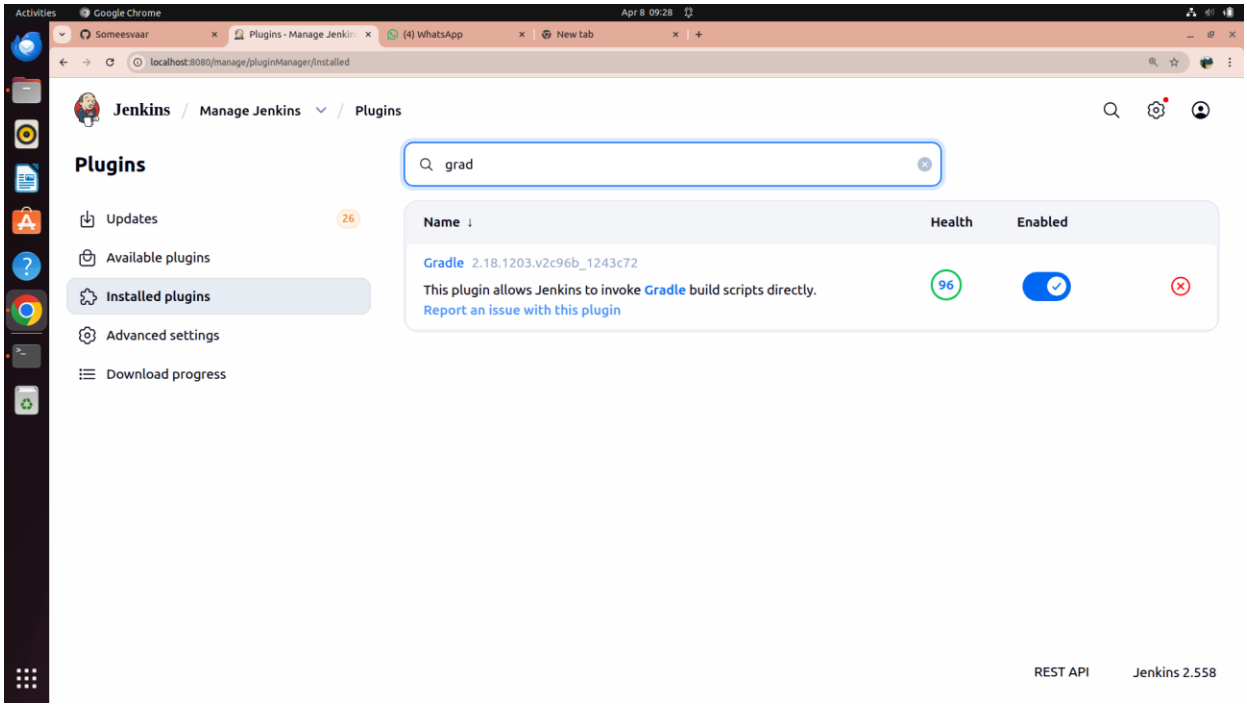

Fig 9: Gradle plugin confirmed as installed and enabled

Step 10: Restart Jenkins

Navigate to **localhost:8080/restart** and click **Yes** to restart Jenkins so all newly installed plugins are properly loaded.

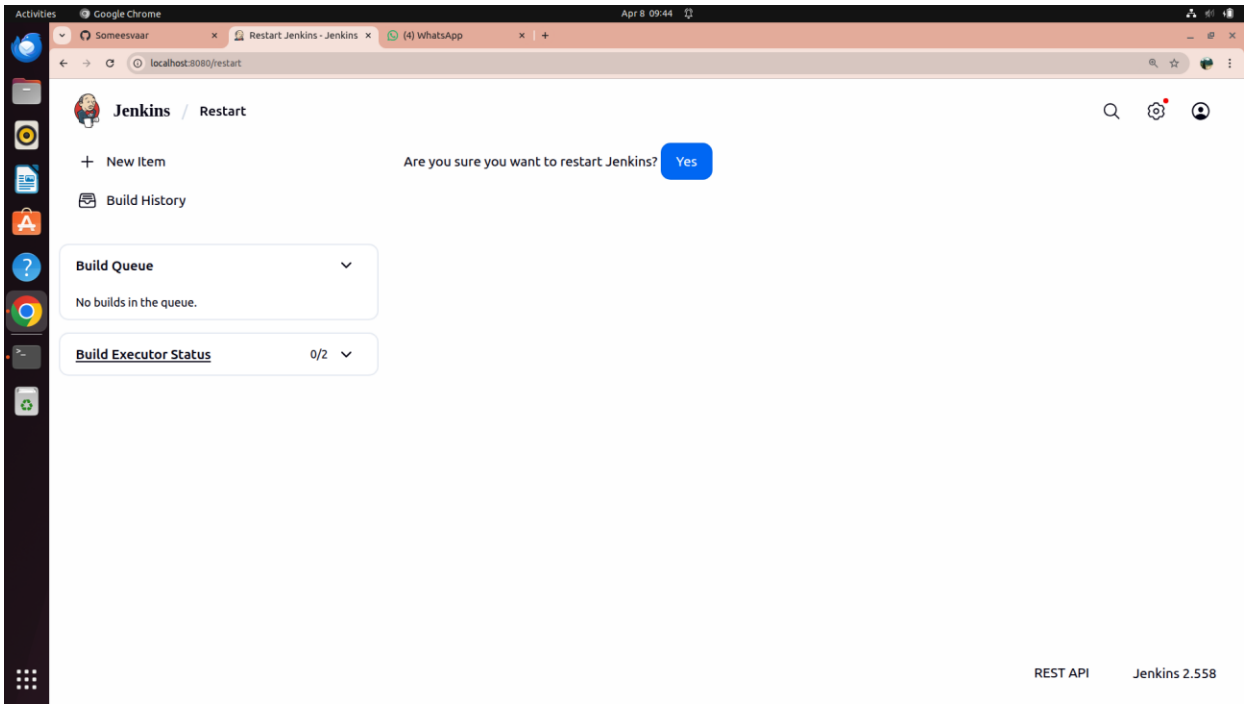

Fig 10: Jenkins restart confirmation prompt

Step 11: Jenkins is Restarting

The browser displays the Jenkins restarting screen. Wait for the browser to automatically reload when Jenkins is ready.

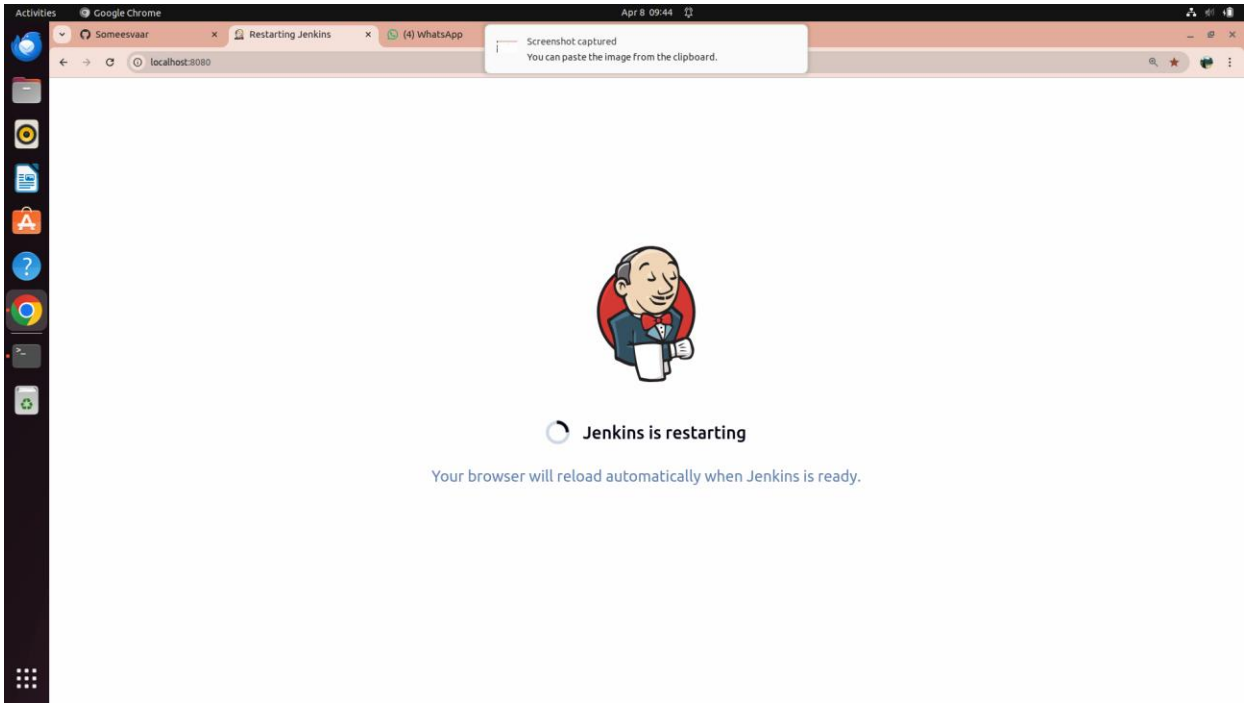

Fig 11: Jenkins restarting screen — browser reloads automatically when ready

Part 2: Configure Global Tools

Step 12: Open Tools Configuration

Navigate to **Manage Jenkins** → **Tools**. This page shows Maven Configuration, JDK installations, Git installations, Gradle installations, and Maven installations. It is used to configure tool paths and automatic installers.

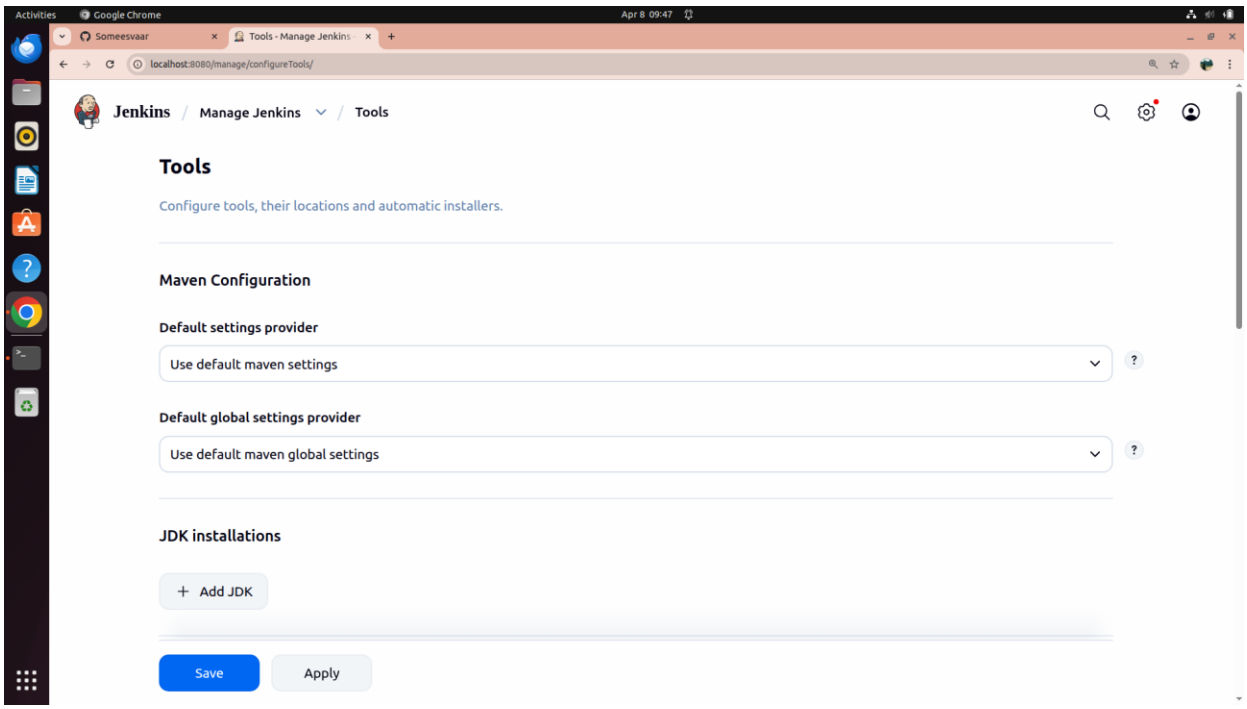

Fig 12: Tools page — Maven Configuration and JDK Installations section

Step 13: Find Tool Paths from Terminal

Open a terminal and run **which java**, **which git**, **which gradle**, and **which mvn** to determine the system paths for each tool:

```
vboxuser@Ubuntu:~$ which java    →  /usr/bin/java
vboxuser@Ubuntu:~$ which git     →  /usr/bin/git
vboxuser@Ubuntu:~$ which gradle  →  /usr/bin/gradle
vboxuser@Ubuntu:~$ which mvn     →  /usr/bin/mvn
```

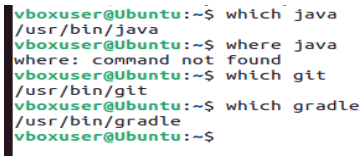

Fig 13: Terminal output showing paths for java, git, gradle, and mvn

```
vboxuser@Ubuntu:~$ which java
/usr/bin/java
vboxuser@Ubuntu:~$ where java
where: command not found
vboxuser@Ubuntu:~$ which git
/usr/bin/git
vboxuser@Ubuntu:~$ which gradle
/usr/bin/gradle
vboxuser@Ubuntu:~$ which mvn
/usr/bin/mvn
vboxuser@Ubuntu:~$
```

Fig 14: Extended terminal output including mvn path

Step 14: Configure JDK Installation

In Tools, click **Add JDK**. Enter **JDK** as the Name and set **JAVA_HOME** to **/lib/jvm/java-11-openjdk-amd64**. Leave "Install automatically" unchecked.

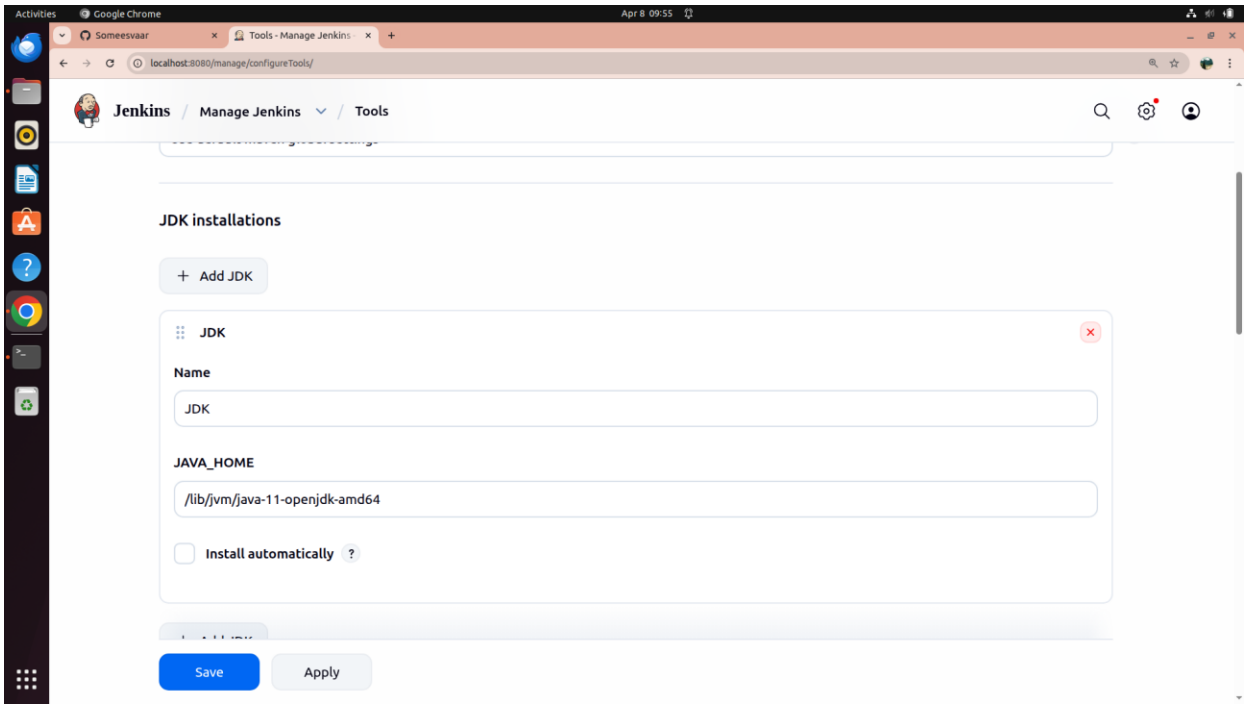

Fig 15: JDK Installation configured with JAVA_HOME set to OpenJDK 11

Step 15: Configure Git Installation

In the **Git installations** section, set Name to **Default** and **Path to Git executable** to **/usr/bin/git**. Leave "Install automatically" unchecked.

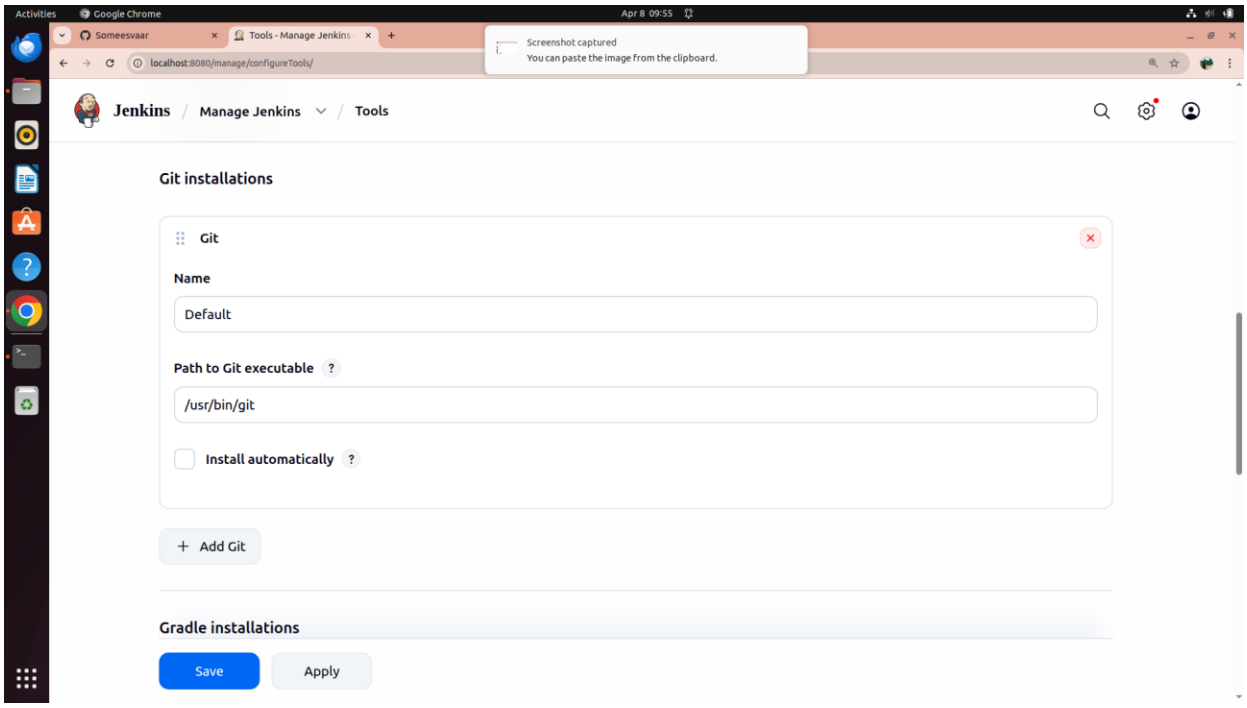

Fig 16: Git Installation configured with path /usr/bin/git

Step 16: Configure Gradle Installation

In the **Gradle installations** section, click Add Gradle. Set Name to **Gradle** and **GRADLE_HOME** to **/usr/bin/gradle**. Leave "Install automatically" unchecked. Click Save.

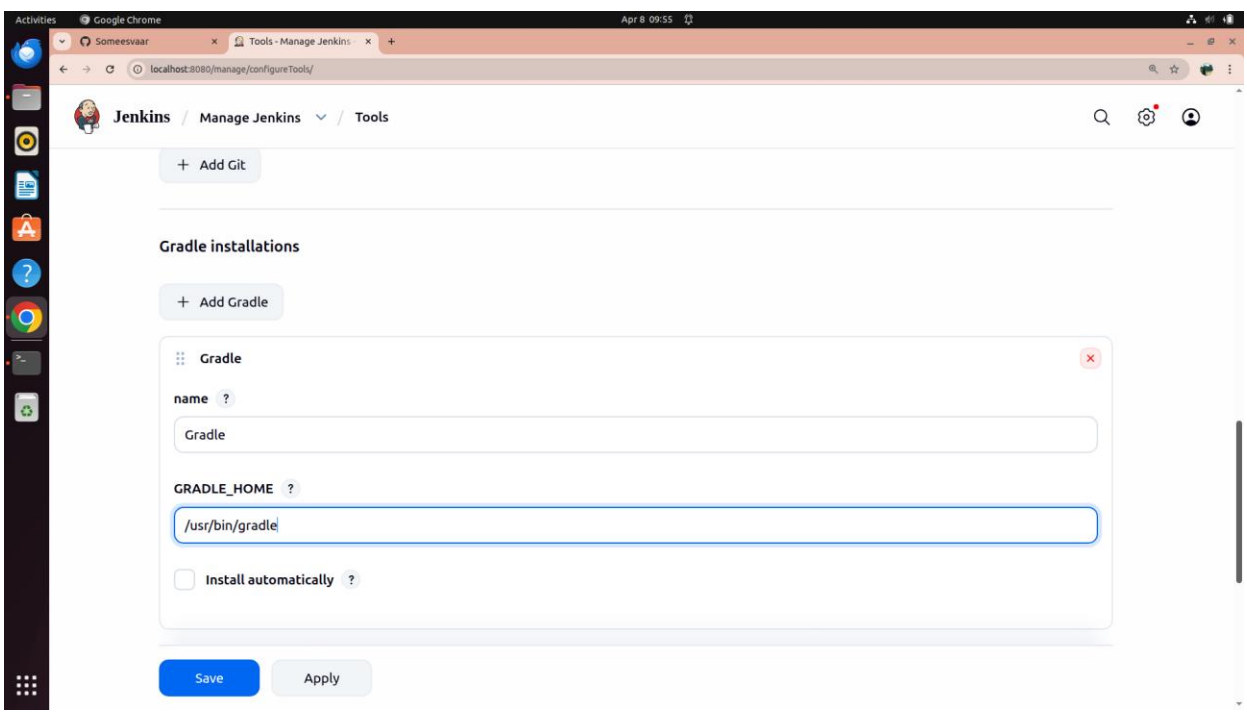

Fig 17: Gradle Installation configured with GRADLE_HOME set to /usr/bin/gradle

Step 17: Configure Maven Installation

In the **Maven installations** section, click Add Maven. Set Name to **Maven** and **MAVEN_HOME** to **/usr/bin/mvn**. Note: Jenkins may show a warning that this path is not a directory on the controller — this is acceptable as it may exist on agents. Click Save.

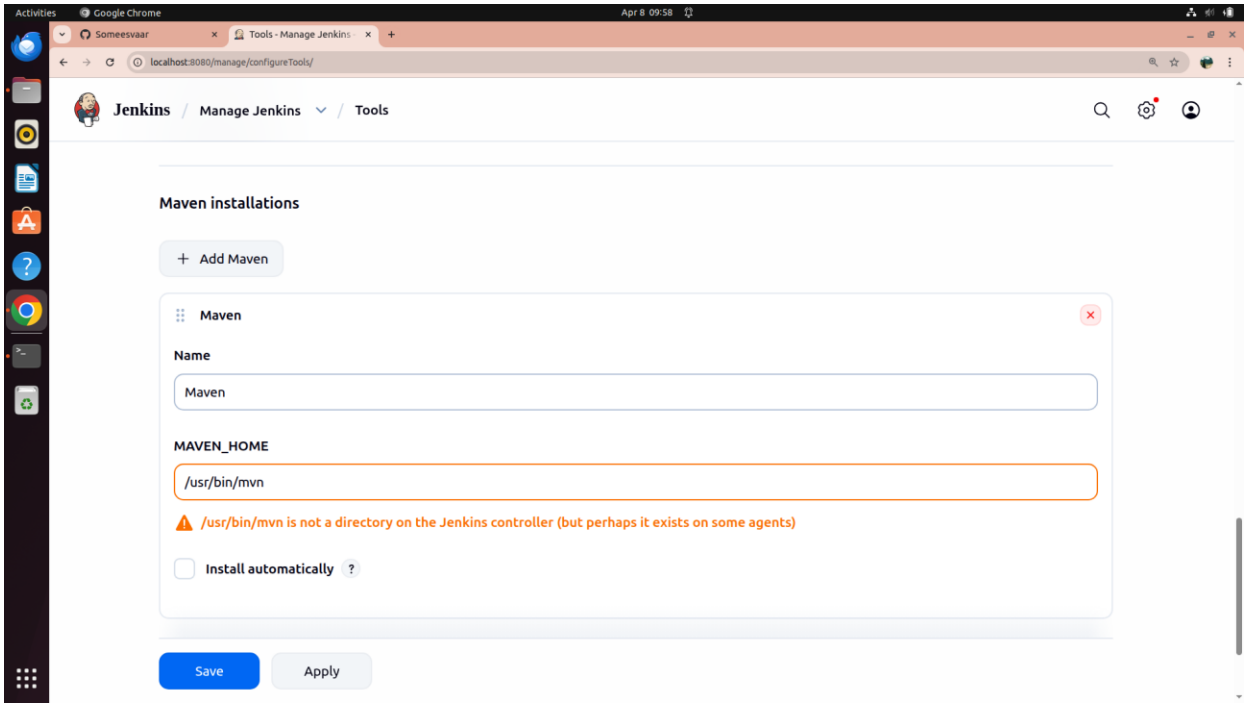

Fig 18: Maven Installation configured with MAVEN_HOME set to /usr/bin/mvn

Part 3: Add GitHub Credentials

Step 18: Open Add Credentials Dialog

Navigate to **Manage Jenkins** → **Credentials** → **System** → **Global**. Click **Add Credentials**. Select the credential type **Username with password** (commonly used for Git, APIs, and registries). Click Next.

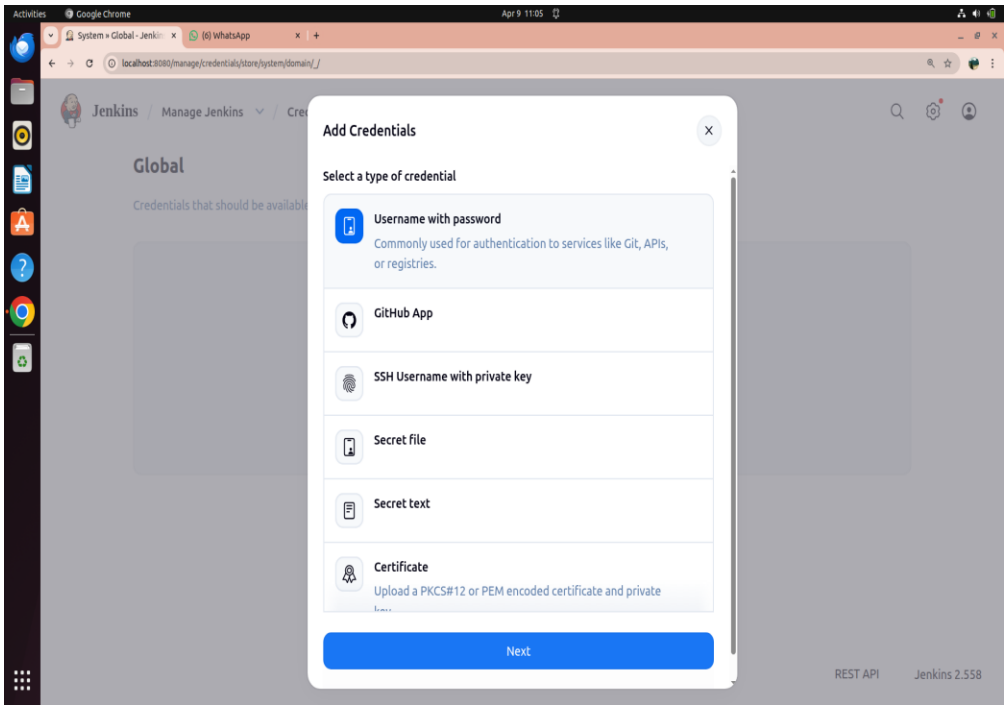

Fig 19: Add Credentials dialog — selecting Username with password type

Step 19: Enter Credential Details

In the Add Username with password form: set **Scope** to **Global**, enter your **Username** (e.g., Someesvaar), enter your **Password** (GitHub Personal Access Token), and set **Description** to **Github Credintials**. Click **Create**.

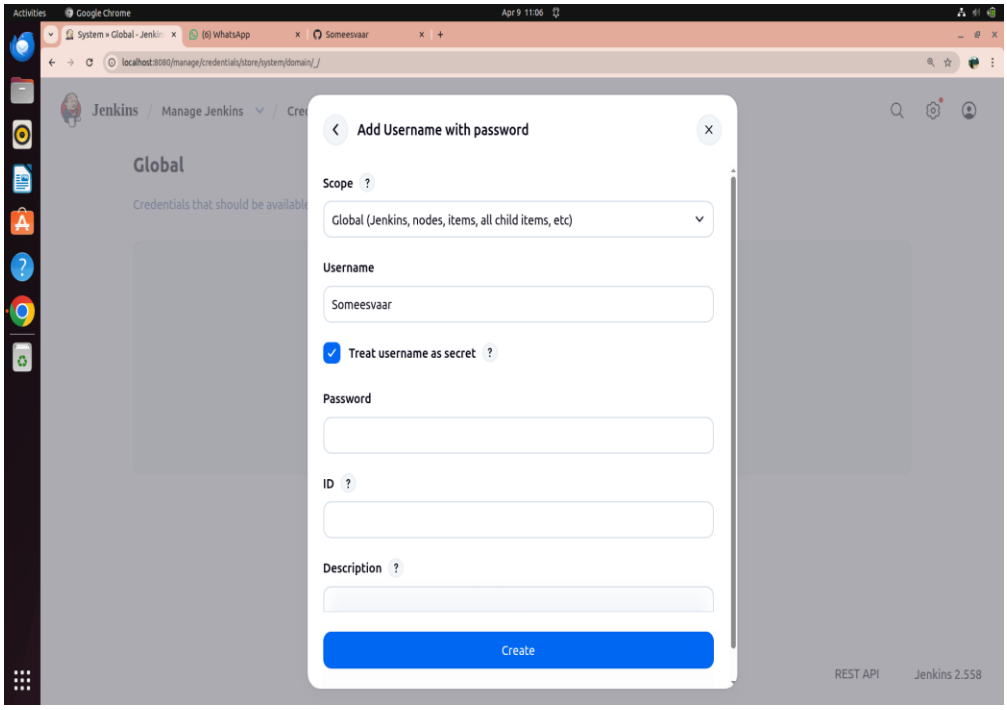

Fig 20: Filling in username, password and description for the GitHub credential

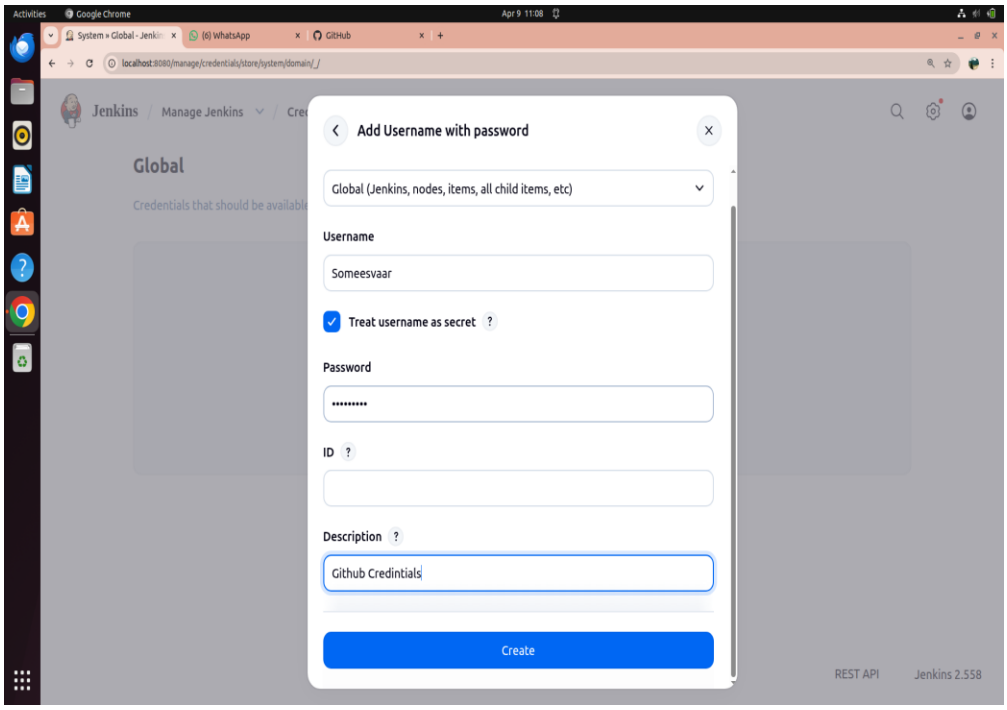

Fig 21: Credential form completed with description "Github Credintials" and password filled

Step 20: Credential Created Successfully

The Global credentials page now shows **Github Credintials** in the list with its ID. The credential is now available for use in all Jenkins pipelines.

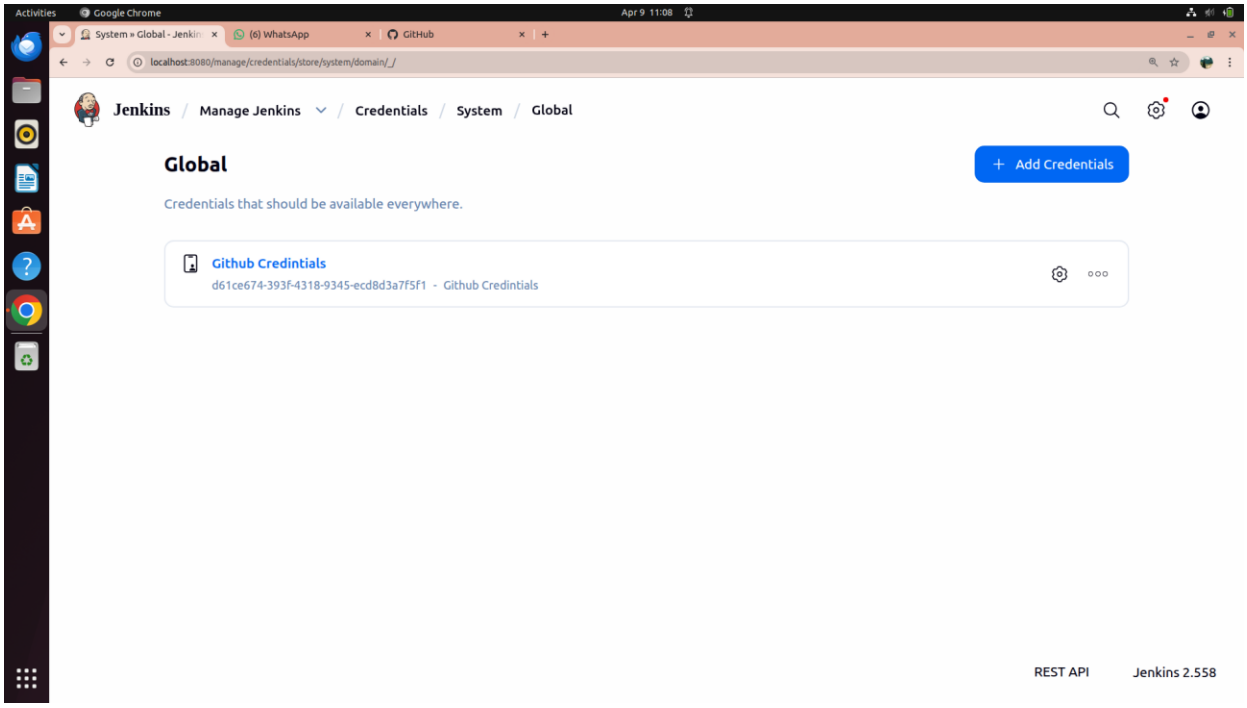

Fig 22: GitHub credential created and visible in the Global credentials list

Part 4: Setting Up Jenkins Pipeline for MyMavenApp

Step 21: Create Jenkinsfile for MyMavenApp

Open the terminal, cd into the **MyMavenApp** folder, and create a **Jenkinsfile** with the following pipeline code. Then commit and push it to GitHub:

```
pipeline {
  agent any  // Use any available agent
  tools {
    maven 'Maven'  // Ensure this matches the name configured in Jenkins
  }
  stages {
    stage('Checkout') {
      steps {
        git branch: 'master', url: 'https://github.com/Someesvaar/MyMavenApp.git'
      }
    }
    stage('Build') {
      steps {
        sh 'mvn clean package'  // Run Maven build
      }
    }
    stage('Test') {
      steps {
        sh 'mvn test'  // Run unit tests
      }
    }
    stage('Run Application') {
      steps {
        sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
      }
    }
  }
  post {
    success { echo 'Build and deployment successful!' }
    failure { echo 'Build failed!' }
  }
}
```

Step 22: Create New Pipeline Job — MavenApp

On the Jenkins homepage, click **New Item**. Enter the item name **MavenApp** and select **Pipeline** as the project type. Click **OK** to proceed to the job configuration page.

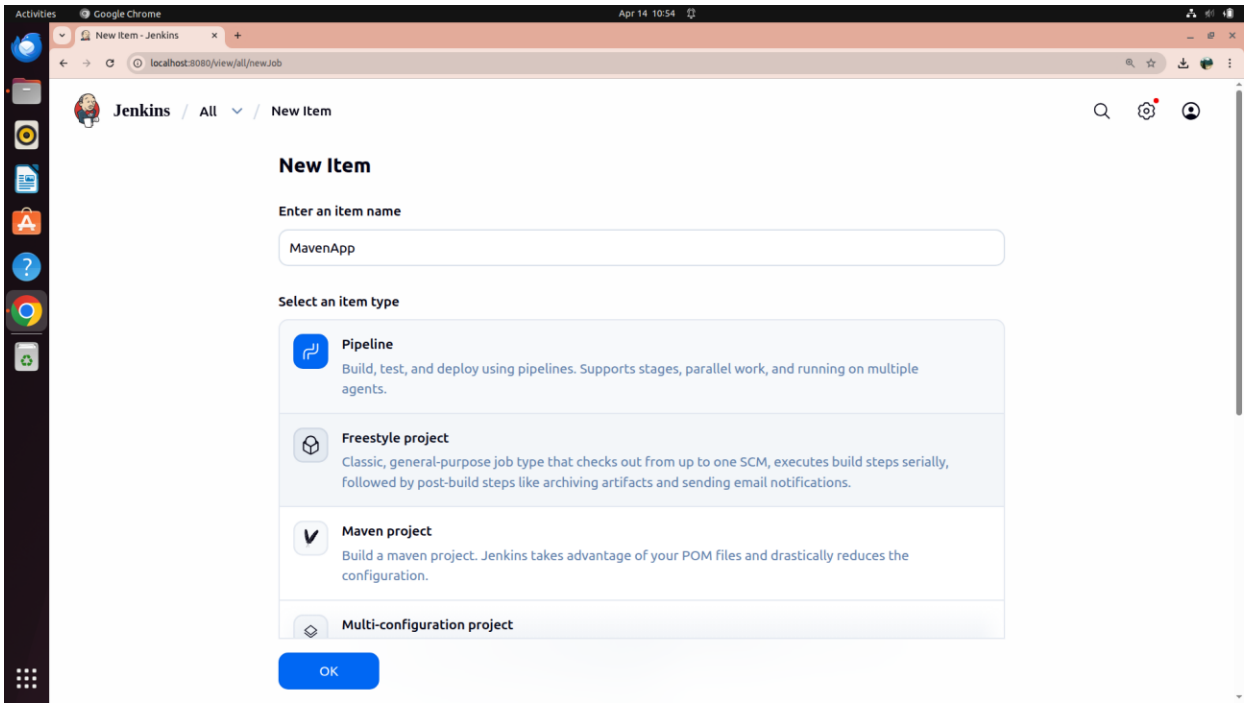

Fig 23: Creating new Jenkins Pipeline job named "MavenApp"

Step 23: Configure Pipeline — Select SCM Definition

In the MavenApp configuration, go to the **Pipeline** section. Under **Definition**, select **Pipeline script from SCM** from the dropdown.

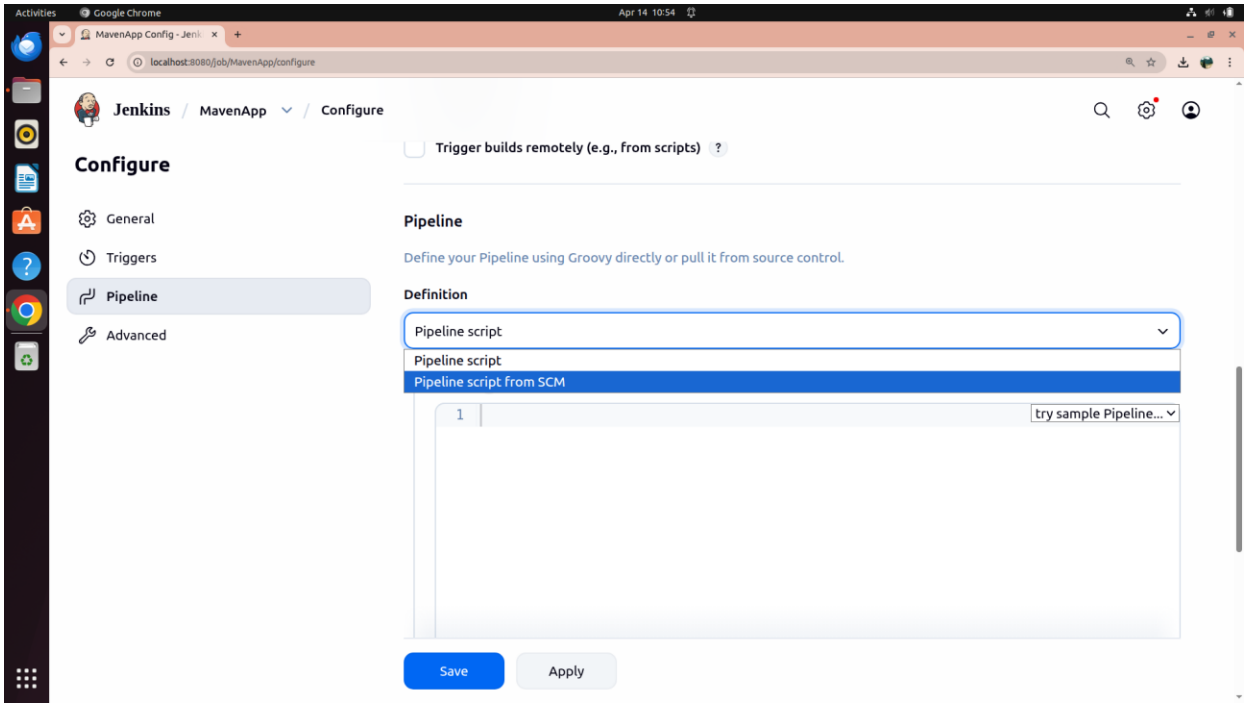

Fig 24: Selecting "Pipeline script from SCM" as the definition type

Step 24: Configure Repository URL and Credentials

Set **SCM** to **Git**. Paste the GitHub URL **https://github.com/Someesvaar/MyMavenApp.git** in **Repository URL**. In the **Credentials** field, select the previously created **Github Credintials**. Set Branch Specifier to ***/master**.

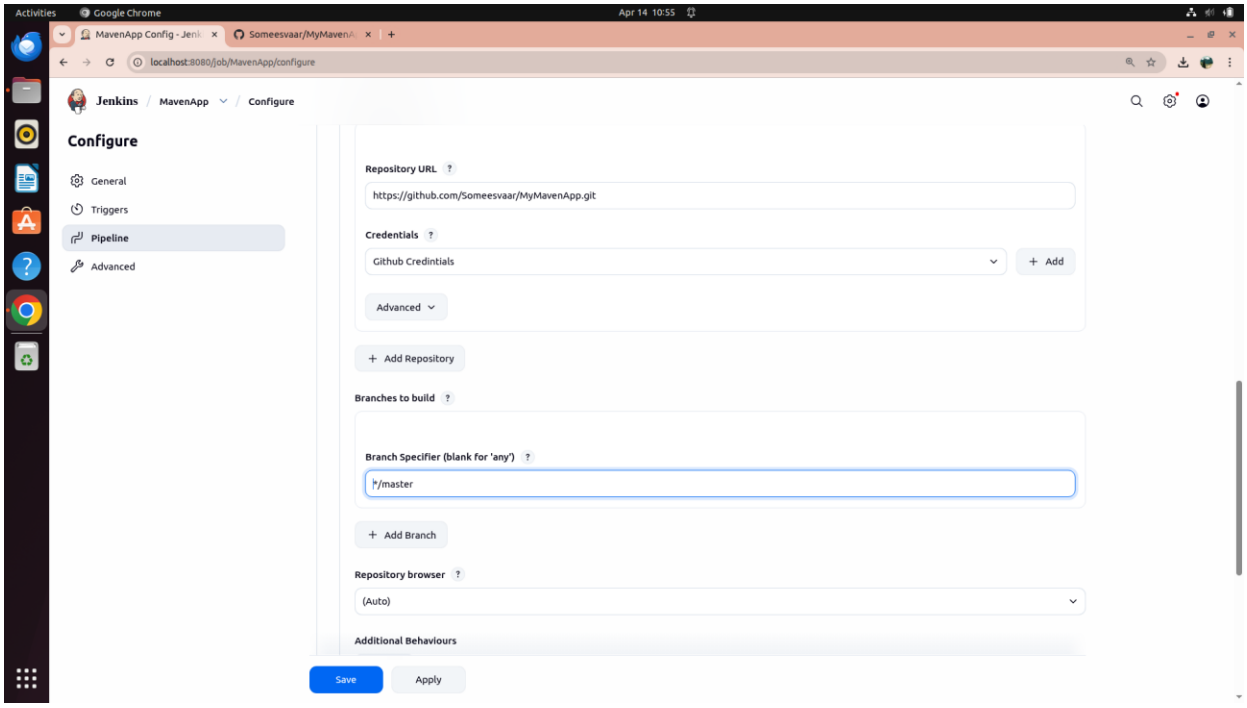

Fig 25: Repository URL and GitHub credentials configured for MyMavenApp

Step 25: Set Script Path and Save

In the **Script Path** field, enter **Jenkinsfile** (the name of the Jenkinsfile created and pushed to GitHub). Click **Apply** and **Save** to finalize the configuration.

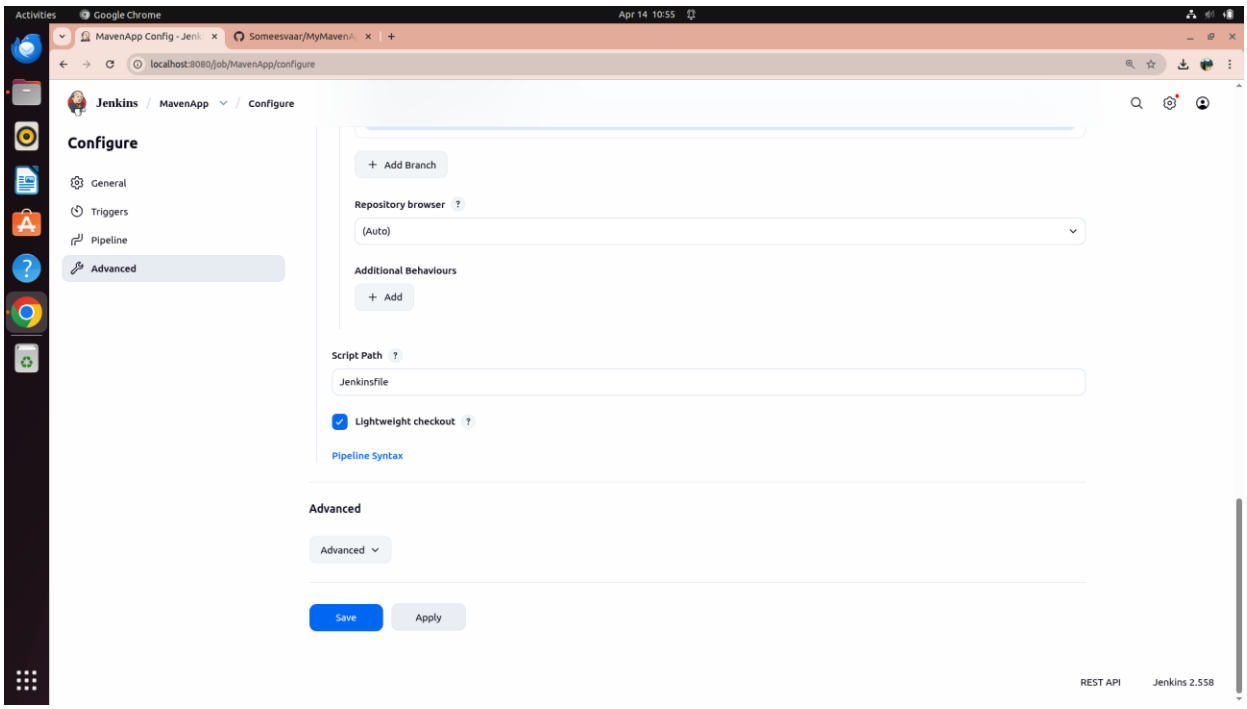

Fig 26: Script Path set to "Jenkinsfile" — Apply and Save to complete configuration

Step 26: Build Now — MyMavenApp Successful

Return to the **MyMavenApp** job and click **Build Now**. The Stage View shows all stages: **Declarative: Checkout SCM**, **Declarative: Tool Install**, **Checkout**, **Build**, **Test**, **Run Application**, and **Declarative: Post Actions** completing successfully (Build #7).

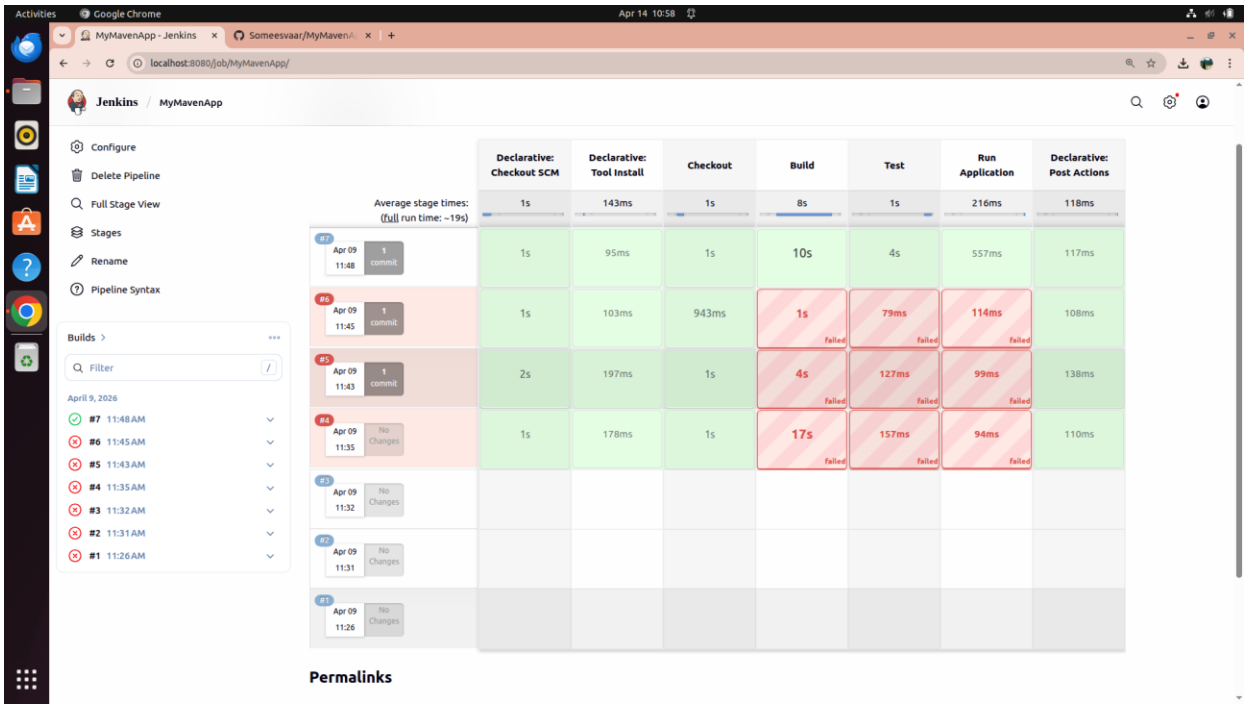

Fig 27: MyMavenApp pipeline — Stage View showing all stages green (Build #7 successful)

Part 5: Setting Up Jenkins Pipeline for MavenToGradle

Step 27: Create New Pipeline Job — MavenToGradle

Back on the Jenkins dashboard, click **New Item** and create a new **Pipeline** job named **MavenToGradle**. This pipeline will run the Maven-to-Gradle migration project.

Step 28: Configure Pipeline SCM — MavenToGradle Repository

In the MavenToGradle pipeline configuration, under **Definition** select **Pipeline script from SCM**. Set SCM to **Git**, paste **https://github.com/Someesvaar/MavenToGradle.git** as the Repository URL, and select **Github Credintials**. Set Branch Specifier to ***/main**. Set Script Path to **Jenkinsfile**.

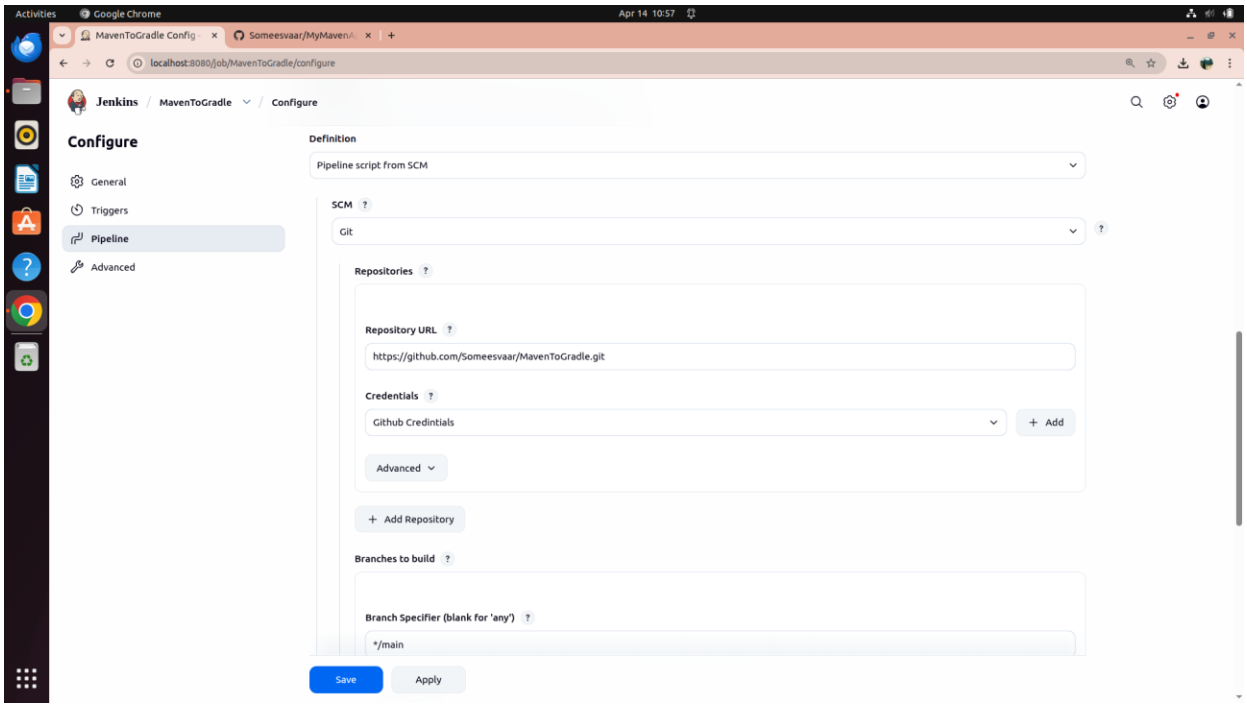

Fig 28: MavenToGradle pipeline configured with repository URL and credentials

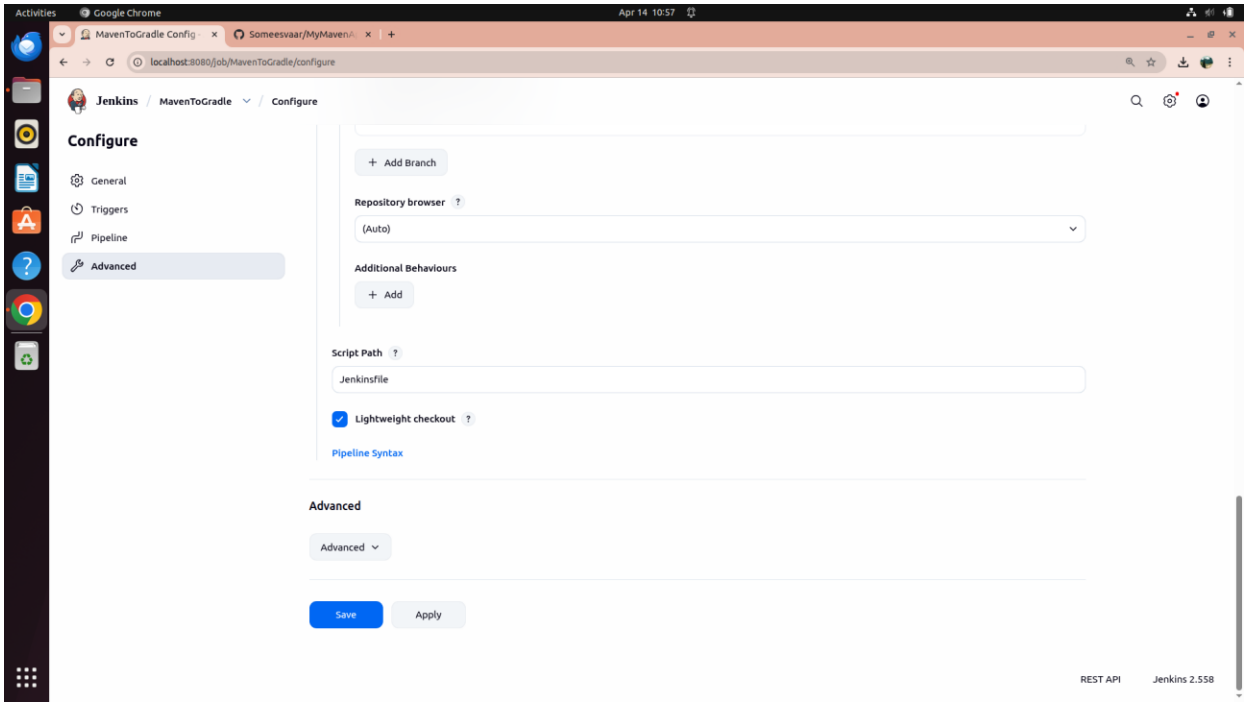

Fig 29: Script Path set to Jenkinsfile for MavenToGradle pipeline

Step 29: MavenToGradle Jenkinsfile

The MavenToGradle Jenkinsfile contains a **Build Maven** stage that builds the original Maven project, and a **Migrate Maven to Gradle** stage that runs **gradle init --type pom** to perform the conversion:

```
pipeline {
  agent any
  tools {
```

```
    maven 'Maven'
    jdk 'JDK'
  }
  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/Someesvaar/MavenToGradle.git'
      }
    }
    stage('Build Maven') {
      steps { sh 'mvn clean package -DskipTests' }
    }
    stage('Migrate Maven to Gradle') {
      steps { sh 'gradle init --type pom' }
    }
  }
  post {
    success { echo 'Migration successful!' }
    failure { echo 'Migration failed!' }
  }
}
```

Step 30: Build Now — MavenToGradle Successful

Return to the **MavenToGradle** job and click **Build Now**. The Stage View shows **Declarative: Checkout SCM**, **Declarative: Tool Install**, **Checkout**, **Build Maven**, **Migrate Maven to Gradle**, and **Declarative: Post Actions** — all completing successfully in ~19 seconds.

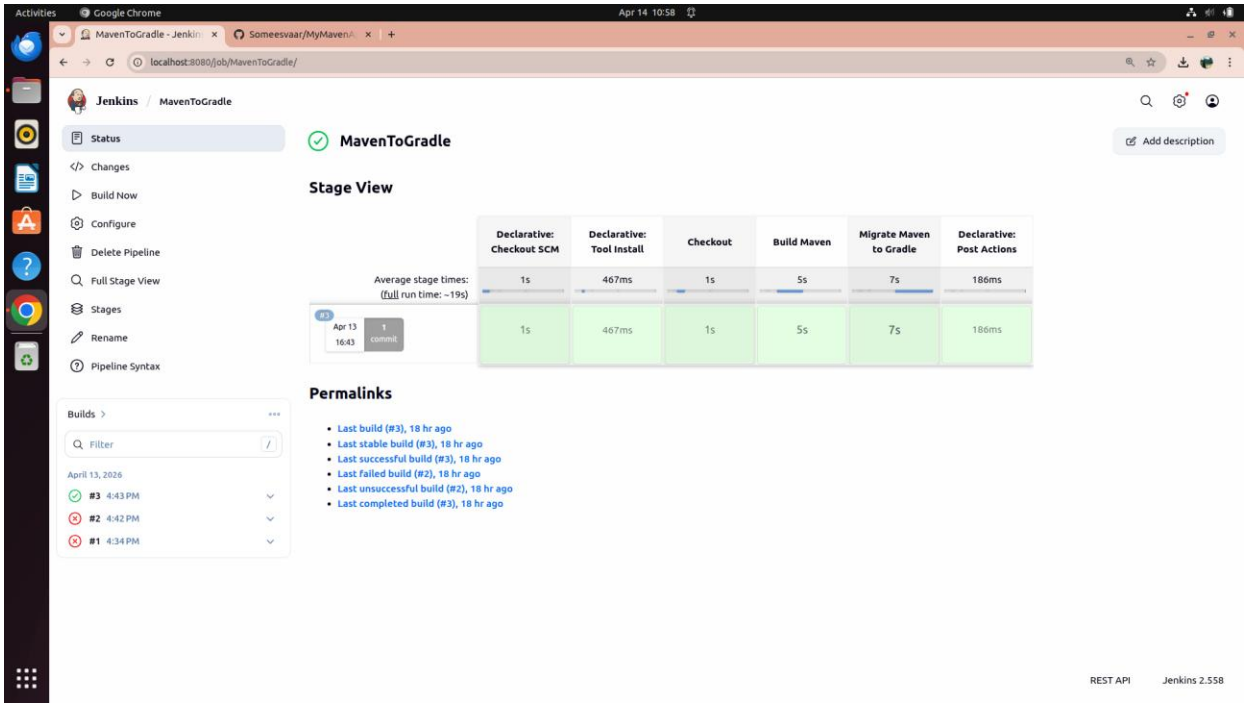

Fig 30: MavenToGradle pipeline — Stage View showing all stages green (Build #3 successful)

Additional: SeleniumApp1 Pipeline Run

A **SeleniumApp1** pipeline job was also configured and run. The Stage View shows build #4 completing successfully with all stages — Checkout SCM, Tool Install, Checkout, Build, Test, Run Application, and Post Actions — passing. Earlier builds #1, #2, #3 show failed stages, demonstrating iterative debugging of the pipeline.

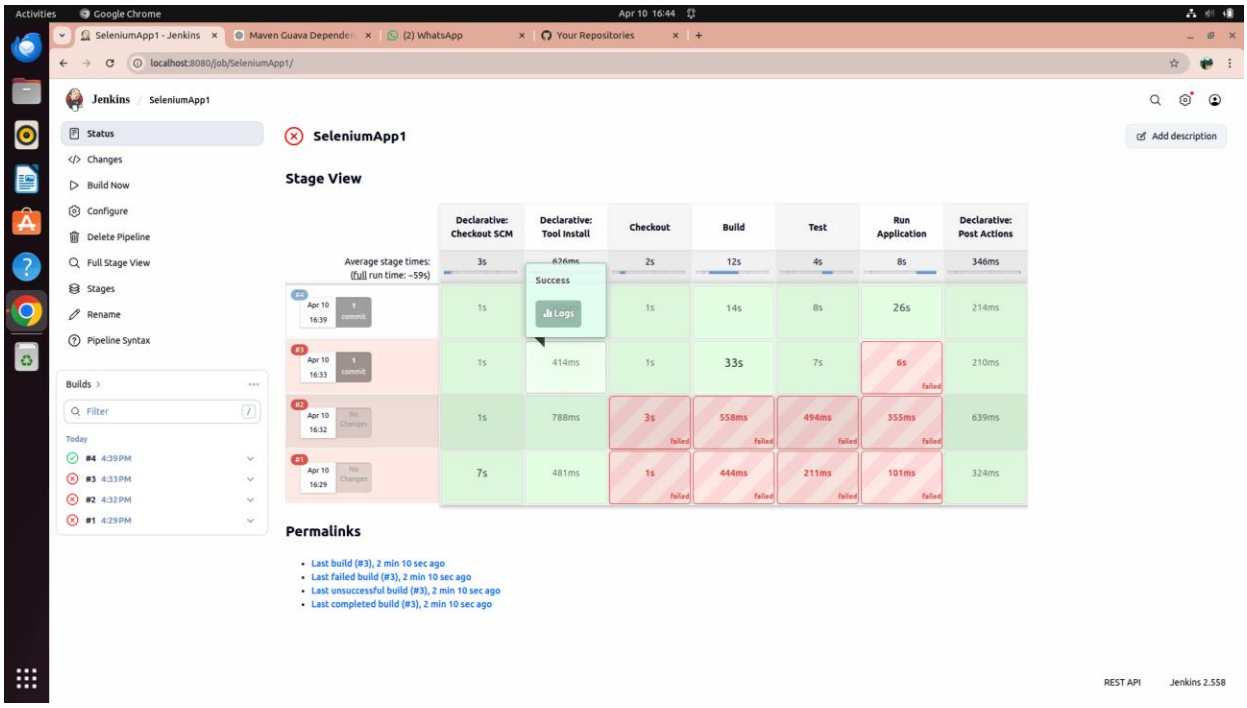

Fig 31: SeleniumApp1 pipeline Stage View — Build #4 succeeds after earlier failures in builds #1–#3